

JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Framework

Pertemuan: Incremental Static Regeneration (ISR)

Topik: Update Halaman Static Tanpa Build Ulang

A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Menjelaskan konsep Incremental Static Regeneration (ISR).
 2. Mengimplementasikan revalidate pada `getStaticProps`.
 3. Menguji pembaruan halaman tanpa build ulang.
 4. Membuat endpoint On-Demand Revalidation.
 5. Mengamankan endpoint revalidation dengan token.
-

B. Dasar Teori

Masalah Static Site Generation (SSG)

Pada SSG:

- Data hanya diambil saat build
 - Perubahan database tidak langsung tampil
 - Harus melakukan `npm run build` ulang
-

Incremental Static Regeneration (ISR)

ISR memungkinkan:

- Halaman tetap static
- Namun dapat diperbarui setelah waktu tertentu
- Tanpa perlu rebuild aplikasi

Alur ISR:

User request

↓

Serve cached static page

↓

Setelah waktu revalidate habis

↓

Next.js fetch ulang data di background

↓

Cache diperbarui

↓

Halaman kembali static

C. Implementasi ISR Otomatis

◆ Bagian 1 – Tambahkan revalidate

- Buka halaman static.tsx pada folder src/pages/produk

```
11.praktikum-Increment_static_regeneration > my-app > src > pages > produk > static.tsx > getStaticProps
1
2 import TampilanProduk from "../../views/product";
3 import { ProductType } from "../../types/Product.type";
4
5
6
Windsurf: Refactor | Explain | Generate JSDoc | X
7 const halamanProdukStatic = (props:{products:ProductType[]}) => {
8   const { products } = props;
9   return (
10     <div>
11       <h1>Halaman Produk Static</h1>
12       <TampilanProduk products={products} />
13     </div>
14   );
15 };
16
17 export default halamanProdukStatic;
18
Windsurf: Refactor | Explain | Generate JSDoc | X
19 export async function getStaticProps() {
20   const res = await fetch('http://127.0.0.1:3000/api/produk');
21   // const response: ProductType[] = await res.json();
22   const response: { data: ProductType[] } = await res.json();
23
24   // console.log("Data produk yang diambil dari API:", response);
25   return {
26     props: {
27       products: response.data,
28     },
29     revalidate: 10, // Revalidate data setiap 10 detik
30   }
31 }
32
```

📌 Artinya:

- Setiap 10 detik halaman akan dicek ulang
 - Jika ada perubahan data → cache diperbarui
-

◆ Bagian 2 – Pengujian ISR

1. Jalankan: (lakukan hal sama seperti JS sebelumnya untuk ngebuild SSG)

- npm run build (jika berhasil)

```
Route (pages)                                Revalidate  Expire
├── /
│   ├── /_app
│   ├── /404
│   ├── /about
│   ├── f /api/[...produk]
│   ├── f /api/hello
│   ├── o /auth/login
│   ├── o /auth/register
│   ├── o /produk
│   ├── ● /produk/[produk] (1695 ms)
│   │   ├── /produk/WoyY5RGELYQslwV5FMlu9 (848 ms)
│   │   └── /produk/pAwIT99SImVbVrNm49m1 (847 ms)
│   ├── o /produk/[produk]_duplicate
│   ├── o /produk/index_Duplicate
│   ├── f /produk/server
│   ├── ● /produk/static (848 ms)
│   ├── o /setting/app
│   ├── o /shop/[...slug]
│   ├── o /user
│   └── o /user/password
└── (Static) prerendered as static content
    ├── (SSG) prerendered as static HTML (uses getStaticProps)
    └── f (Dynamic) server-rendered on demand
```

- npm run start

2. Tambahkan data baru di database pada firebase

Add a document

Parent path
/products

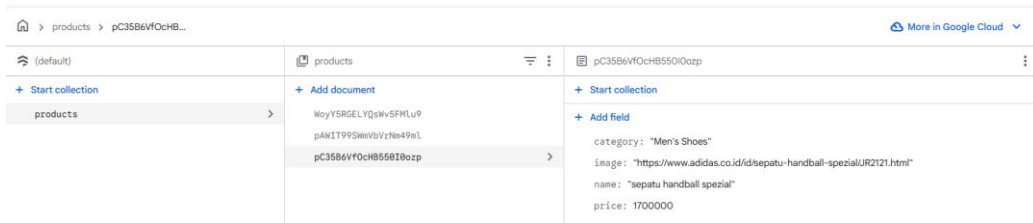
Document ID

pC35B6VfOcHB550I0ozp

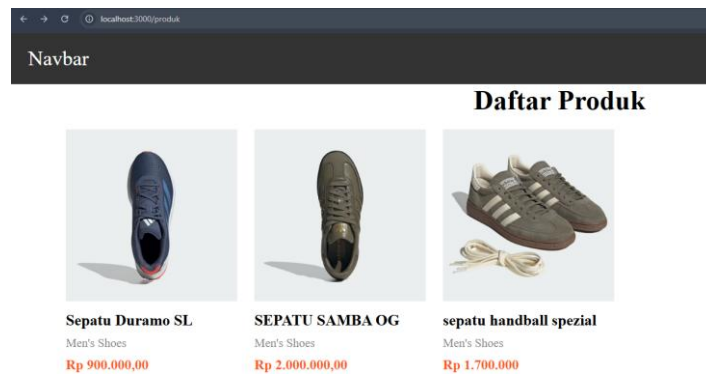
Field	Type	Value
name	string	sepatu handball spesial
price	number	1700000
image	string	https://www.adidas.co.id/id/sepatu-handball-spezial/UR2121.html
category	string	Men's Shoes

... Add field

Cancel Save



3. Refresh halaman sebelum 10 detik → Data lama.
 - Sebelum 10 detik data yang akan ditampilkan masih data lama
4. Refresh setelah 10 detik → Data baru muncul.



D. On-Demand Revalidation

Jika tidak ingin menunggu waktu revalidate, gunakan endpoint khusus.

◆ Bagian 1 – Buat API Revalidate

- Buat file revalidate.ts pada folder pages/api/ dan modifikasi

```
11.praktikum-Increment_static_regeneration > my-app > src > pages > api > TS revalidate.ts > ...
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiRequest, NextApiResponse } from "next";
3
4 type Data = {
5   revalidated: boolean;
6 };
7
8 export default async function handler(
9   req: NextApiRequest,
10  res: NextApiResponse<Data>,
11 ) {
12   try {
13     await res.revalidate("/produk/static");
14     return res.status(200).json({ revalidated: true });
15   } catch (error) {
16     console.error("Error in API route:", error);
17     res.status(500).send({ revalidated: false });
18   }
19 }
20
```

Note : jika kita menggunakan code ini ada sedikit masalah yaitu Ketika salah satu data dihapus maka data yang terhapus tidak secara otomatis terupdate pada layar meskipun sudah di refresh selama 10 detik. Untuk bisa terupdate data yang ditampilkan diperlukan trigger dengan mengakses apinya <http://localhost:3000/api/produk>

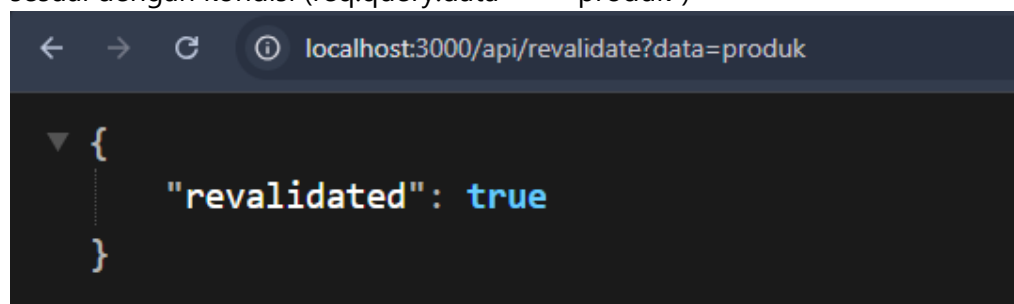
◆ Bagian 2 – Tambahkan Parameter Data

Untuk mengatasi hal tersebut (pada bagian 1) maka suatu kondisi pada file revalidate.ts

- Modifikasi file revalidate.ts

```
11.praktikum-Increment_static_regeneration > my-app > src > pages > api > TS revalidate.ts > ...
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse, NextApiRequest } from "next";
3
4 type Data = {
5   revalidated: boolean;
6   message?: string;
7 };
8
9 export default async function handler(
10   req: NextApiRequest,
11   res: NextApiResponse<Data>,
12 ) {
13   if (req.query.data === "produk") {
14     try {
15       await res.revalidate("/produk/static");
16       return res.status(200).json({ revalidated: true });
17     } catch (error) {
18       console.error("Error in API route:", error);
19       res.status(500).send({ revalidated: false });
20     }
21   }
22
23   return res.json({
24     revalidated: false,
25     message: "Invalid query parameter. Expected 'data=produk'.",
26   });
27 }
28
```

- Uji coba menambahkan parameter dan value pada url <http://localhost:3000/api/revalidate?data=produk> maka akan muncul true dan sesuai dengan kondisi (req.query.data === "produk")



- Uji coba dengan url <http://localhost:3000/api/revalidate?data=>

```
localhost:3000/api/revalidate?data=
{
  "revalidated": false,
  "message": "Invalid query parameter. Expected 'data=produk'."
}
```

◆ Bagian 3 – Tambahkan Token Security

Perlu ditambahkan paramater agar user tidak merubah data melalui url

- Buka file .env dan modifikasi

```
11.praktikum-Increment_static_regeneration > my-app > .env.local
1  FIREBASE_API_KEY=AIzaSyBWR40s6R6v-P407FBUclw9FhZLhw9PbSM
2  FIREBASE_AUTH_DOMAIN=framework-next-54b28.firebaseio.com
3  FIREBASE_PROJECT_ID=framework-next-54b28
4  FIREBASE_STORAGE_BUCKET=framework-next-54b28.firebaseio.com
5  FIREBASE_MESSAGING_SENDER_ID=4519320556
6  FIREBASE_APP_ID=1:4519320556:web:711cee20e042981378fec9
7
8  REVALIDATE_TOKEN=12345678
```

- Modifikasi file revalidate.ts tambahkan kondisi pada line 13 - 17

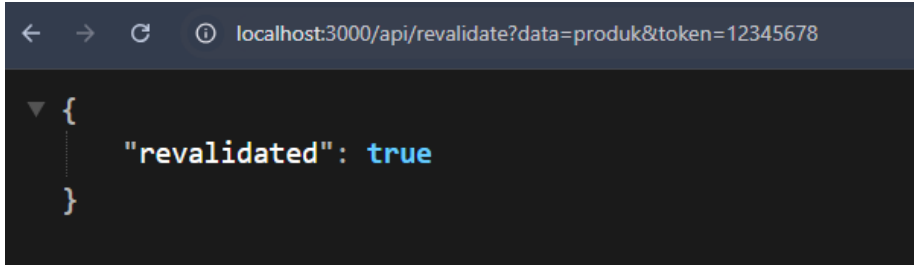
```
11.praktikum-Increment_static_regeneration > my-app > src > pages > api > TS revalidate.ts > handler
1  // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2  import type { NextApiRequest, NextApiResponse } from "next";
3
4  type Data = {
5    revalidated: boolean;
6    message?: string;
7  };
8
9  export default async function handler(
10    req: NextApiRequest,
11    res: NextApiResponse<Data>,
12  ) {
13    if (req.query.token !== process.env.REVALIDATE_TOKEN) {
14      return res.status(401).json({
15        revalidated: false,
16        message: "Insert correct token",
17      });
18    }
19
20    if (req.query.data === "produk") {
21      try {
22        await res.revalidate("/produk/static");
23        return res.status(200).json({ revalidated: true });
24      } catch (error) {
25        console.error("Error in API route:", error);
26        res.status(500).send({ revalidated: false });
27      }
28    }
29
30    return res.json({
31      revalidated: false,
32      message: "Invalid query parameter. Expected 'data=produk'.",
33    });
34  }
35
```

E. Pengujian Manual Revalidation

Akses:

<http://localhost:3000/api/revalidate?data=products&token=12345678>

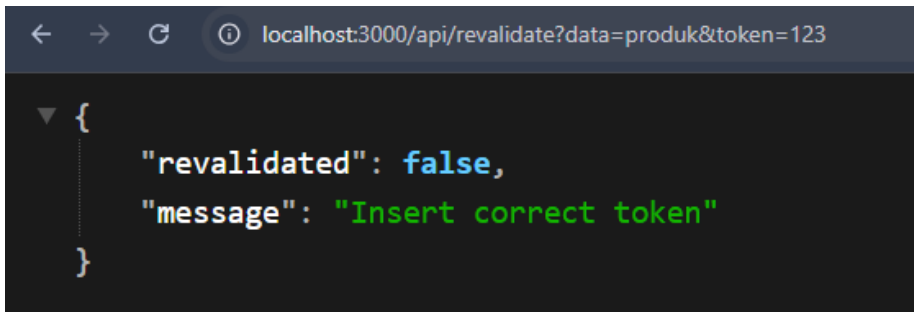
Jika benar:



```
localhost:3000/api/revalidate?data=produk&token=12345678
{
  "revalidated": true
}
```

{ revalidate: true }

Jika token salah:



```
localhost:3000/api/revalidate?data=produk&token=123
{
  "revalidated": false,
  "message": "Insert correct token"
}
```

F. Perbandingan SSG vs ISR

Aspek	SSG	ISR
Update Data	Harus build ulang	Otomatis / Trigger
Cache	Static permanen	Static + Refresh
Cocok untuk	Konten tetap	Konten semi-dinamis

G. Tugas Praktikum

Tugas Individu

1. Tambahkan lagi produk pada firebase
2. Implementasikan ISR dengan **revalidate: 10**.
3. Tambahkan endpoint On-Demand Revalidation.
4. Tambahkan validasi token.

5. Uji dengan:
- Token benar
 - Token salah
 - Tanpa token
-

 H. Pertanyaan Analisis

1. Mengapa ISR lebih fleksibel dibanding SSG?
2. Apa perbedaan revalidate waktu dan on-demand?
3. Mengapa endpoint revalidation harus diamankan?
4. Apa risiko jika token tidak digunakan?
5. Kapan ISR lebih cocok dibanding SSR?